SOURCE-BACKED COMPANION

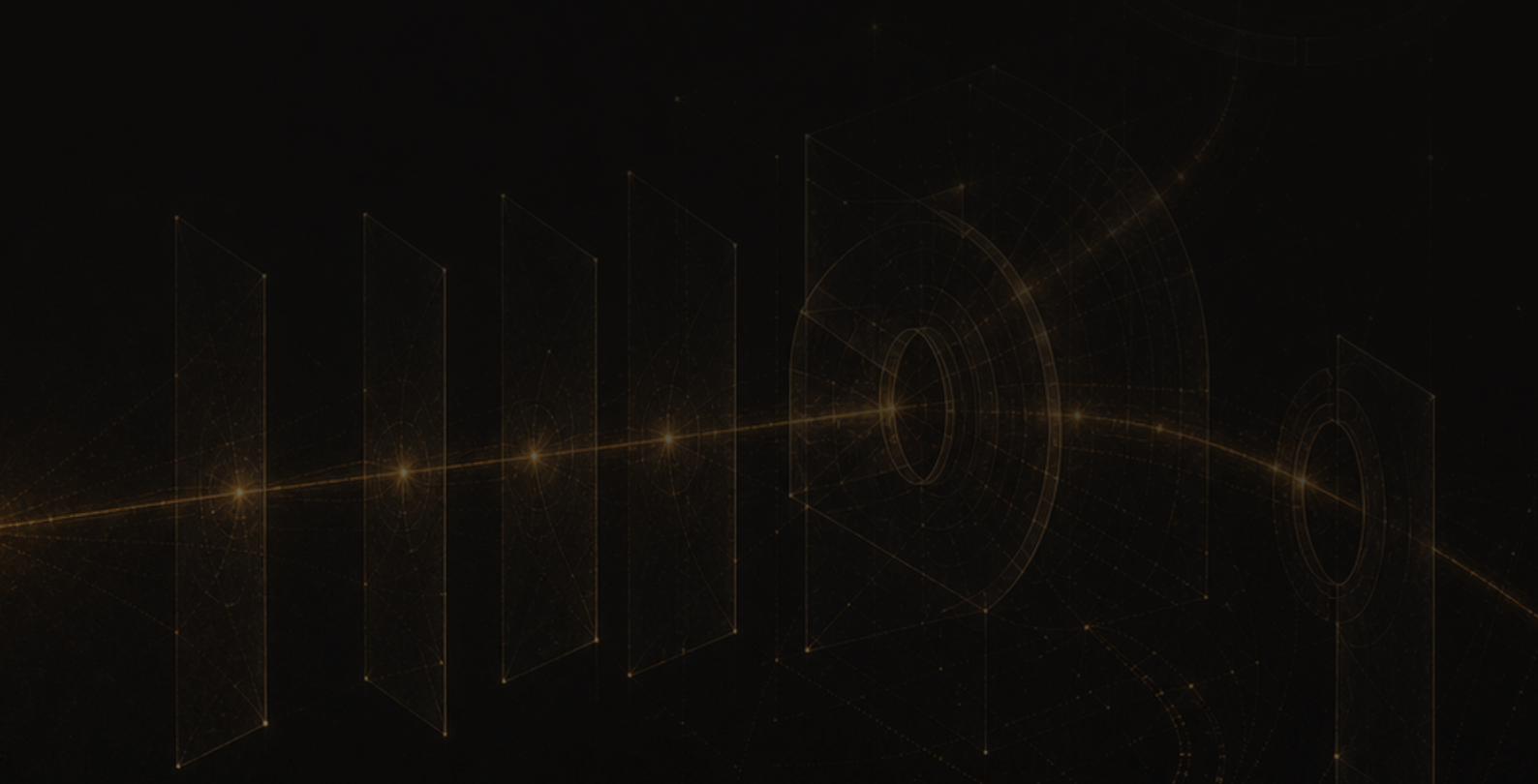
AIRLOCK PROGRAMS

Structured Unix work between agents and the world.

THE AUTHORING RAIL

CONTRACT MODEL

```
ActionCall
    -> PlanDraft
    -> Admission
    -> ExecutionAuthority
    -> Capture | Invoke | Apply | RequestExternal
```



MACOS FIRST / AGENT ONLY

USABLE DEVELOPER PREVIEW / 2026-07-30

00 CAVEAT

HOW TO READ THIS BOOK

These pages show current program syntax without upgrading a developer preview into a shell-replacement claim.

SOURCE / DESIGN.md / README.md / docs/macos-v1.md

> **AIRLOCK PROGRAMS EXPRESS STRUCTURED UNIX WORK. EXISTING UNIX PROGRAMS STILL DO THE WORK.**

RUNNABLE

Checked-in current syntax or CLI surface. It may still require bindings, policy, fixtures, or a supported macOS profile.

CONTRACT MODEL

Implemented typed architecture, shown as data or a rail. It is not Airlock source syntax.

INTEGRATION SKETCH

Current language forms aimed at real OpenShell or Vouch work. The remote integration has not run end to end.

CAVEAT

A visible boundary on what the example proves. Compatibility is broad, native containment is narrow, and receipts are not omniscience.

01 **CAPTURE**

OBSERVATION

Information enters.

02 **INVOKE**

COMPUTATION

Existing code runs.

03 **APPLY**

MUTATION

Managed state changes.

04 **REQUEST EXTERNAL**

EMISSION

Intent stages.

THE TWO LAWS

Only the Reaper unlinks. Explicit profiles may narrow authority; agent code cannot widen it.

WHAT IS NOT CLAIMED

No confidentiality claim for native-contained. No VM equivalence. No broad shell replacement claim yet.

EVIDENCE / The generated plate is atmosphere. Every readable claim and code block is deterministic.

01 RUNNABLE

INSTALL. PROBE. INSPECT.

Start from a checkout, build the paired command surfaces, then ask Airlock to describe itself.

SOURCE / README.md: Install from a checkout

docs/programs/snippets/14-install.sh RUNNABLE

```
bun install
bun run verify
bun run build:macos
sh scripts/install-macos.sh
export PATH="$HOME/.local/bin:$PATH"

airlock doctor
airlock actions
airlock schema plan
airlock-agent actions
```

SUPERVISOR SURFACE

`airlock` retains policy, raw maintenance, Hold, Outbox commit/cancel, undo, and reap authority.

AGENT SURFACE

`airlock-agent` exposes programs, schemas, capabilities, and read-only state inspection — not terminal authority.

> THE HARNESS GIVES THE AGENT AIRLOCK-AGENT, NOT A PEER SHELL TOOL.

EVIDENCE / Current release requirement: macOS and Bun 1.3.11 or newer. The install prefix must be on PATH.

02 RUNNABLE

YOUR FIRST AIRLOCK PROGRAM

The executable is one field. Every argument is a separate atom. The runtime receives no command string.

SOURCE / src/actions/Catalog.ts / src/process/Process.ts

docs/programs/snippets/01-first.air RUNNABLE

```
let result = process.run({
  executable: "/usr/bin/printf",
  args: ["%s\n", "hello from Airlock"],
  cwd: workspace,
  stdin: "discard",
  stdout: "capture",
  stderr: "capture",
  timeoutMs: 5000,
  cellProfile: "compatibility"
})

assert result.state == "succeeded", "printf failed"
return result
```

SUPERVISOR INVOCATION RUNNABLE

```
airlock run hello.air \
  --workspace /tmp \
  --bindings '{"workspace":"/tmp"}'
```

NO IMPLICIT SHELL

Airlock invokes `[executable, ...args]`. Metacharacters inside an argument remain literal at this seam.

AMBIENT HOST AUTHORITY

This example selects compatibility. Child reads, writes, sends, helpers, config, and descriptors are not contained.

EVIDENCE / Structured invocation removes shell parsing from orchestration; it does not make compatibility a sandbox.

03 RUNNABLE

A SMALL LANGUAGE AROUND EFFECTS

Bindings supply values. Pure control chooses when to request actions. Neither can mint authority.

SOURCE / src/language/ast.ts / src/language/evaluator.ts

docs/programs/snippets/02-control.air RUNNABLE

```
assert enabled, "the supervisor must enable this workload"

for entry in writes {
  let written = file.write({
    path: entry.path,
    content: entry.content
  })
  assert written.state == "applied", "write failed"
}

if expected == 3 {
  return { state: "complete", iterations: expected }
} else {
  return { state: "unexpected-input", iterations: expected }
}
```

FINITE
List iteration snapshots a finite list. Range loops are half-open and bounded.

TYPED FAILURE
`assert` stops later work and preserves completed action evidence.

NO WIDENING
Bindings change data, never the supervisor-selected profile or grants.

AGENT TOOL CALL RUNNABLE

```
airlock-agent run docs/programs/snippets/02-control.air \
  --workspace /tmp/airlock-demo \
  --bindings '{
    "enabled": true,
    "expected": 3,
    "writes": [
      {"path":"a.txt","content":"a"},
      {"path":"b.txt","content":"b"},
      {"path":"c.txt","content":"c"}
    ]
  }'
```

EVIDENCE / The evaluator has one injected world seam: ActionResolver.resolve(action, input).

04 CONTRACT MODEL

FOUR EFFECTS. ONE CLOSED PLAN.

Actions lower into an implemented candidate Plan algebra before Admission binds any authority.

SOURCE / src/plan/Plan.ts / docs/contracts/plan-runtime.md

THE CLOSED ALGEBRA CONTRACT MODEL

```
1 PlanNode =
2     Capture
3   | Invoke
4   | Apply
5   | RequestExternal
```

01 CAPTURE

file.read / glob / stat

Provenance attaches at entry.

02 INVOKE

process.run

A selected profile runs existing code.

03 APPLY

file.write / move / remove

Hold performs the live transition.

04 REQUEST EXTERNAL

http.stage

Outbox retains intent before dispatch.



IMPLEMENTED CANDIDATE SEAM

The four nodes and total lowering exist. Representative completeness across Unix work remains a falsifiable hypothesis.

ONLY TWO LAWS

The Plan algebra is not promoted into a law. The Reaper and ratchet remain the only frozen invariants.

EVIDENCE / A PlanDraft has unresolved requirements. ExecutionAuthority is the Runtime input.

05 RUNNABLE

CAPTURE THE WORKSPACE

Read, inspect, list, glob, and stat are explicit observations with resource requirements and artifacts.

SOURCE / src/actions/Catalog.ts / examples/parity/filesystem.air

docs/programs/snippets/03-capture.air RUNNABLE

```
let readme = file.read({
  path: "README.md",
  format: "text"
})

let sources = file.glob({
  root: ".",
  pattern: "**/*.ts"
})

let metadata = file.stat({
  path: "README.md"
})

return {
  readme: readme,
  sources: sources,
  metadata: metadata
}
```

ACTIONS	PLAN NODE	DEFAULT REALM
5	Capture	local
inspect / read / list / glob / stat	observation enters	still admitted explicitly

WHAT PROVENANCE BUYS

Downstream artifacts can point back to the admitted locator and resource identity used for capture.

WHAT IT DOES NOT BUY

Native-contained currently permits ambient host reads. Observation evidence is not a confidentiality boundary.

EVIDENCE / Current formats: text, bytes, and JSON. Following symlinks in file.stat is rejected.

06 RUNNABLE

INVOKE WITHOUT COMMAND CONSTRUCTION

Literal argv replaces shell parsing at the Airlock boundary - including strings that look hostile to a shell.

SOURCE / examples/parity/process-pipeline.air / src/process/Process.ts

docs/programs/snippets/04-invoke-literal.air RUNNABLE

```
let result = process.run({
  executable: "/usr/bin/printf",
  args: [
    "%s",
    "literal; touch should-not-exist"
  ],
  cwd: workspace,
  stdin: "discard",
  stdout: "capture",
  stderr: "capture",
  outputLimitBytes: 65536,
  cellProfile: "compatibility"
})

assert result.state == "succeeded", "printf failed"
return result
```



STRUCTURED STREAMS

stdin is discard, inherit, inline text, or an ArtifactId. stdout and stderr are capture, discard, or inherit.

AMBIENT HOST AUTHORITY

The literal string is safe from implicit shell parsing, but the compatibility child remains broadly empowered.

EVIDENCE / An explicitly admitted interpreter may still interpret its own input. Airlock does not infer intent.

07 RUNNABLE

A PIPELINE IS AN ARTIFACT GRAPH

Captured stdout becomes named input to the next Invoke. No pipe syntax or intermediary shell process is required.

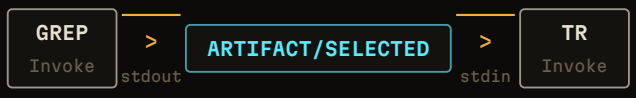
SOURCE / examples/parity/process-pipeline.air / test/parity-process.test.ts

docs/programs/snippets/05-pipeline.air RUNNABLE

```
let selected = process.run({
  executable: "/usr/bin/grep",
  args: ["-n", "needle", "input.txt"],
  cwd: workspace,
  stdin: "discard",
  stdout: "capture",
  stderr: "capture",
  cellProfile: "compatibility"
})
assert selected.state == "succeeded", "grep failed"

let upper = process.run({
  executable: "/usr/bin/tr",
  args: ["a-z", "A-Z"],
  cwd: workspace,
  stdin: {
    kind: "artifact",
    id: selected.stdout_artifact.id
  },
  stdout: "capture",
  stderr: "capture",
  cellProfile: "compatibility"
})
assert upper.state == "succeeded", "tr failed"

return {
  selected: selected.stdout,
  transformed: upper.stdout
}
```



EXPLICIT DATAFLOW

The artifact id carries bytes and provenance. Process order follows declared dependency. This is not an atomic pipeline.

PROOF RECEIPT

Executed output: `2:needle second` became `2:NEEDLE SECOND`; both displayed Invokes succeeded.

EVIDENCE / Output and the final returned value remain bounded by configured limits.

08 RUNNABLE

UNIX PROGRAMS STAY UNIX PROGRAMS

Airlock does not implement archive semantics. It gives `/usr/bin/tar` a structured invocation and bounded authority.

SOURCE / examples/parity/corpus/archive-roundtrip.air

docs/programs/snippets/06-tar.air RUNNABLE

```
let packed = process.run({
  executable: "/usr/bin/tar",
  args: ["-czf", "package.tgz", "package"],
  cwd: workspace,
  env: {
    COPYFILE_DISABLE: "1",
    LANG: "C"
  },
  stdin: "discard",
  stdout: "capture",
  stderr: "capture",
  timeoutMs: 30000,
  cellProfile: "native-contained"
})
assert packed.state == "succeeded", "archive creation failed"

let archive = file.inspect({
  path: workspace + "/package.tgz"
})

return { packed: packed, archive: archive }
```

TAR archive

GIT versioning

SED editing

MAKE build

SQLITE3 database

PRISTINE COMPONENT

The generic action is `process.run`. Tool definitions may improve ergonomics but cannot add authority or Plan constructors.

NATIVE BOUNDARY

Native-contained supports a narrower file/directory workload. Descendant executables must be declared.

EVIDENCE / The archive algorithm remains tar's. Airlock owns admission, execution bounds, finality, and evidence.

09 RUNNABLE

APPLY THROUGH HOLD

Managed file actions perform admitted local changes. Hold displaces prior bindings before live state changes.

SOURCE / src/Hold.ts / examples/parity/filesystem.air

docs/programs/snippets/07-apply.air RUNNABLE

```
let directory = file.mkdir({
  path: "generated",
  parents: true
})

let written = file.write({
  path: "generated/report.txt",
  content: "ready\n"
})

let copied = file.copy({
  source: "generated/report.txt",
  destination: "generated/report.backup.txt"
})

let moved = file.move({
  source: "generated/report.txt",
  destination: "report.txt"
})

let removed = file.remove({
  path: "generated/report.backup.txt"
})

return {
  directory: directory,
  written: written,
  copied: copied,
  moved: moved,
  removed: removed
}
```



LAW 01

Only the Reaper unlinks. Managed removal or replacement first retains prior state through Hold.

RECOVERY BOUNDARY

Undo is bounded by retained material, supported filesystem semantics, and conflict checks. Multi-entry ACID is not claimed.

EVIDENCE / Compatibility-child syscalls do not become Apply and receive no Hold guarantee.

10 PROOF

LET RM BE RM – REMOVE FINALITY INSTEAD

The destructive executable runs inside the private view. Its deletion becomes a delta; Hold owns the live transition.

SOURCE / examples/parity/native-destructive.air / test/parity-native-contained.test.ts

docs/programs/snippets/08-rm-rf.air RUNNABLE

```
let removed = process.run({
  executable: "/bin/rm",
  args: ["-rf", "protected"],
  cwd: workspace,
  stdin: "discard",
  stdout: "capture",
  stderr: "capture",
  timeoutMs: 30000,
  cellProfile: "native-contained"
})

assert removed.state == "succeeded", "contained removal failed"
return removed
```



THE USEFUL CAPABILITY REMAINS. IN THIS SUPPORTED NATIVE-CONTAINED WORKLOAD, THE IRREVERSIBLE PATH DOES NOT.

EXECUTED CORPUS

The destructive native workload removes a directory through the private Cell, merges the delta, and restores it with supervisor undo.

NOT SYSCALL INTERCEPTION

Running `/bin/rm -rf` in compatibility would retain ambient host mutation authority and sit outside this recovery envelope.

EVIDENCE / This is the clearest example of Airlock physics around an existing Unix tool.

11 CAVEAT

PROFILES NARROW

With no policy a program works in its workspace and nowhere else. A supervisor's policy opens more, and a grant can only narrow the ceilings it sets; agent code cannot select a profile.

SOURCE / src/host/FilesystemHost.ts / docs/usage.md

COMPATIBILITY

IMPLEMENTED

BROAD, NEVER SELF-SELECTED

Existing host authority, configuration, descendants, descriptors, filesystem, and network remain available. A compatibility process is not confined, so it may only run when a person approves it.

NATIVE-CONTAINED

IMPLEMENTED

EXPLICIT CONFINEMENT

Private workspace, live-write denial, network denial, delta generation, and Hold-backed Apply for the supported subset.

VM-ENCLOSED

FUTURE

STRONGER FUTURE ENCLOSURE

Unavailable today. The CLI refuses this profile rather than silently falling back to compatibility.

THE POLICY CARRIES THE PROFILE

RUNNABLE

```
airlock-agent describe --workspace .

airlock review job.air
airlock approve-plan <plan id>

airlock-agent run job.air \
  --workspace /absolute/workspace \
  --plan <plan id>
```

DENY BY DEFAULT

With no policy, a program reads and changes files in its workspace only; a process or the network needs a policy that grants it. Missing enforcement fails closed.

NATIVE IS NOT CONFIDENTIAL

The current Seatbelt profile permits ambient host reads and does not claim complete execution closure or VM equivalence.

EVIDENCE / A confined process merges back through Hold; an unconfined one always waits for a person.

12 RUNNABLE

ADMISSION OWNS AUTHORITY

One policy per session: tool grants under shared resource ceilings, a budget, and named flow rules.

SOURCE / src/core/session/ToolPolicy.ts / docs/usage.md

docs/programs/snippets/18-policy.json RUNNABLE

```json
{
  "schemaVersion": "airlock/tool-policy/v1",
  "toolGrants": [
    { "id": "write", "tool": "file.write", "when": "now" },
    { "id": "build", "tool": "process.run", "when": "now", "host": { "profile": "native-contained" } },
    { "id": "hook", "tool": "http.stage" }
  ],
  "resources": {
    "path": ["/absolute/workspace"],
    "executable": { "roots": ["/usr/bin/touch"], "edges": [] },
    "endpoint": ["https://hooks.example.com/build"]
  },
  "budget": { "maxCalls": 64, "maxInputBytes": 1000000 }
}
```



SCHEMA-FIRST

The policy, requirements, grants, handles, resolutions, and execution authority are decoded contracts - not CLI folklore.

NO AUTHORITY MINTING

Program source and definitions cannot mint grants. This example policy authorizes only `/usr/bin/touch` in its listed workspace.

EVIDENCE / Runtime revalidates closure and grant lifetime at each node boundary.

13 RUNNABLE

COMPUTE IN A PRIVATE VIEW

Native-contained runs the executable away from the live workspace, derives a supported delta, then asks Hold to apply it.

SOURCE / src/cell/Cell.ts / docs/macos-v1.md

docs/programs/snippets/09-native-touch.air RUNNABLE

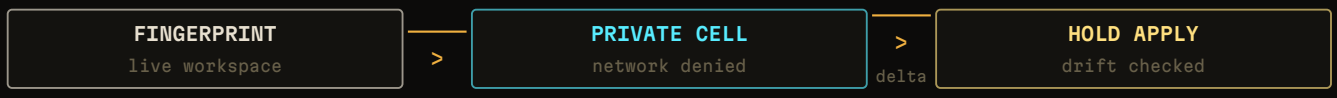
```
let created = process.run({
  executable: "/usr/bin/touch",
  args: ["created.txt"],
  cwd: workspace,
  stdin: "discard",
  stdout: "capture",
  stderr: "capture",
  timeoutMs: 30000,
  cellProfile: "native-contained"
})

assert created.state == "succeeded", "contained create failed"
return created
```

docs/programs/snippets/15-native-run.sh RUNNABLE

```
AIRLOCK_POLICY_FILE=/absolute/policy.json \
  airlock run /absolute/create.air \
  --workspace /absolute/workspace \
  --profile native-contained \
  --bindings '{"workspace":"/absolute/workspace"}'

airlock held
airlock undo
```



NARROW EVIDENCE ENVELOPE

Current native merge covers supported top-level regular-file or directory deltas. Hardlinks, special files, live foreign writers, ACL/xattr fidelity, and multi-entry atomicity remain outside the claim.

EVIDENCE / Unavailable native enforcement never falls back to compatibility.

14 RUNNABLE

STAGE BEFORE THE WIRE

RequestExternal creates durable local intent. A later supervisor action decides whether to cancel or dispatch.

SOURCE / src/Outbox.ts / src/outbox/Contract.ts

docs/programs/snippets/10-http-stage.air RUNNABLE

```
let staged = http.stage({
  endpoint: endpoint,
  method: "POST",
  headers: {
    "content-type": "application/json"
  },
  body: "{\"operation\":\"snapshot-ready\"}",
  holdMillis: 300000
})

assert staged.state == "staged"
return staged
```

docs/programs/snippets/16-outbox.sh RUNNABLE

```
airlock pending

# Cancel while the intent is still staged:
airlock cancel emi_...

# Or let the supervisor cross the wire:
airlock commit emi_...
```



ONLY COMMIT DISPATCHES

Airlock-owned HTTP intent touches the wire only inside Outbox.commit. Node success means staged, not sent.

HONEST UNCERTAINTY

After a possible dispatch, ambiguity becomes `uncertain`; it is never automatic permission to retry.

EVIDENCE / A person commits or cancels; nothing crosses the wire without that decision. There is no flush and no daemon.

15 INTEGRATION SKETCH

VOUCH PATH I: SNAPSHOT

OpenShell remains OpenShell. Python and SQLite retain their application semantics. Airlock structures the host orchestration.

SOURCE / examples/vouch/snapshot.air / ../vouch/assets/box-runtime-v1.py

docs/programs/snippets/11-vouch-snapshot.air:1-21 INTEGRATION SKETCH

```
let snapshot = process.run({
  executable: openshell,
  args: [
    "sandbox", "exec",
    "--name", sandbox_name,
    "--no-tty",
    "--timeout", "120",
    "--",
    "/usr/bin/python3", "-I", "-S",
    "/opt/airlock-programs/sqlite-safe-snapshot.py",
    "--state-root", hermes_home,
    "--output", remote_archive
  ],
  cwd: workspace,
  stdin: "discard",
  stdout: "capture",
  stderr: "capture",
  timeoutMs: 150000,
  cellProfile: "compatibility"
})
assert snapshot.exit_code == 0, "snapshot failed"
```

STRUCTURED CONTROLLER CALL

OpenShell is one admitted executable. Its subcommand, sandbox name, timeout, remote executable, and helper arguments stay distinct.

COMPATIBILITY – AMBIENT

OpenShell owns the remote effects; Outbox does not mediate them. Local process cancellation is not remote operation cancellation.

EVIDENCE / Snapshot phase. The existing Python helper retains SQLite and archive semantics.

15B INTEGRATION SKETCH

VOUCH SNAPSHOT: BRING THE ARTIFACT HOME

A second structured invocation downloads the remote artifact. A managed observation records local archive metadata.

SOURCE / examples/vouch/snapshot.air / examples/vouch/OPERATIONS.md

docs/programs/snippets/11-vouch-snapshot.air:23-40 INTEGRATION SKETCH

```
23 let downloaded = process.run({
24   executable: openshell,
25   args: [
26     "sandbox", "download",
27     sandbox_name,
28     remote_archive,
29     local_archive
30   ],
31   cwd: workspace,
32   stdin: "discard",
33   stdout: "capture",
34   stderr: "capture",
35   timeoutMs: 180000,
36   cellProfile: "compatibility"
37 })
38 assert downloaded.exit_code == 0, "download failed"
39
40 return file.inspect({ path: local_archive })
```



GENERIC PHYSICS

Invoke existing controller. Capture process evidence. Inspect the result. No Vouch-shaped Airlock primitive appears.

REMOTE PROOF MISSING

The source parses and lowers today. A real OpenShell/Vouch snapshot run remains unproved.

EVIDENCE / Airlock does not claim the remote artifact is correct merely because download exited zero.

16A INTEGRATION SKETCH

VOUCH PATH II: STAGE RESTORE INPUT

Inspect local evidence first, then upload with a structured controller invocation and an explicit deadline.

SOURCE / examples/vouch/restore.air / examples/vouch/OPERATIONS.md

docs/programs/snippets/12-vouch-restore.air:1-20 INTEGRATION SKETCH

```
let archive = file.inspect({ path: local_archive })
assert archive.bytes > 0, "restore archive is empty"

let uploaded = process.run({
  executable: openshell,
  args: [
    "sandbox", "upload",
    sandbox_name,
    local_archive,
    remote_archive,
    "--no-git-ignore"
  ],
  cwd: workspace,
  stdin: "discard",
  stdout: "capture",
  stderr: "capture",
  timeoutMs: 120000,
  cellProfile: "compatibility"
})
assert uploaded.exit_code == 0, "upload failed"
```

PRECONDITION IS VISIBLE

The archive must exist and contain bytes before the upload request can begin.

COMPATIBILITY - AMBIENT

OpenShell owns upload behavior and remote effects. Outbox does not mediate this controller traffic.

EVIDENCE / Local timeout bounds the controller process. It does not prove remote cancellation.

16B INTEGRATION SKETCH

VOUCH RESTORE: EXECUTE THE EXISTING HELPER

The embedded Python command becomes literal argv to an existing helper, with captured streams and an explicit postcondition.

SOURCE / examples/vouch/restore.air / ../vouch/assets/box-runtime-v1.py

docs/programs/snippets/12-vouch-restore.air:22-44 INTEGRATION SKETCH

```
22 let restored = process.run({
23   executable: openshell,
24   args: [
25     "sandbox", "exec",
26     "--name", sandbox_name,
27     "--no-tty",
28     "--timeout", "180",
29     "--",
30     "/usr/bin/python3", "-I", "-S",
31     "/opt/airlock-programs/safe-restore.py",
32     "--archive", remote_archive,
33     "--destination", state_home
34   ],
35   cwd: workspace,
36   stdin: "discard",
37   stdout: "capture",
38   stderr: "capture",
39   timeoutMs: 210000,
40   cellProfile: "compatibility"
41 })
42
43 assert restored.exit_code == 0, "restore failed"
44 return restored
```

WHAT GETS SIMPLER

Executable, argv, deadline, stdout, stderr, and the exit postcondition stay visible instead of hiding inside shell text.

WHAT AIRLOCK DOES NOT OWN

Archive collision policy, SQLite correctness, OpenShell transport, and helper correctness remain existing-program semantics.

EVIDENCE / A remote `/usr/bin/python3` argv atom is not a local descendant executable edge.

17A INTEGRATION SKETCH

VOUCH PATH III: PRESERVE BEFORE REPLACE

The replacement route begins by running the snapshot program and refusing to continue if that phase fails.

SOURCE / examples/vouch/replace.air / docs/vouch-first.md

docs/programs/snippets/13-vouch-replace.air:1-15 INTEGRATION SKETCH

```
let snapshot = process.run({
  executable: airlock_program,
  args: [
    "run", snapshot_program,
    "--sandbox", sandbox_name,
    "--archive", local_archive
  ],
  cwd: workspace,
  stdin: "discard",
  stdout: "capture",
  stderr: "capture",
  timeoutMs: 300000,
  cellProfile: "compatibility"
})
assert snapshot.exit_code == 0, "snapshot failed"
```



FAILURE IS A STOP

The assertion prevents later backup and staging requests after a failed snapshot result.

STILL AMBIENT

The nested Airlock controller invocation uses compatibility in this sketch. Its internal remote effects are not contained here.

EVIDENCE / This page proves program shape only, not a completed Vouch replacement.

17B — INTEGRATION SKETCH

VOUCH REPLACE: RETAIN, THEN STAGE

Local recovery and remote finality stay separate: retain a backup through Apply, then stage replacement intent.

SOURCE / examples/vouch/replace.air / docs/vouch-first.md

docs/programs/snippets/13-vouch-replace.air:17-35 INTEGRATION SKETCH

```
17 let saved = file.copy({
18   source: local_archive,
19   destination: durable_backup
20 })
21 assert saved.state == "applied", "backup was not retained"
22
23 let replacement = http.stage({
24   method: "POST",
25   endpoint: realm_endpoint,
26   holdMillis: 30000,
27   body: "{\"action\":\"replace-machine\"}"
28 })
29 assert replacement.state == "staged"
30
31 return {
32   snapshot: snapshot,
33   saved: saved,
34   replacement: replacement
35 }
```



THIS PROGRAM DOES NOT REPLACE ANYTHING

There is no commit, completion check, health gate, restore, endpoint broker, or remote Plan transport in the snippet.

THE IMPORTANT SEPARATION

Hold can recover managed local backup state. It cannot undo a remote machine replacement after the recipient observes it.

EVIDENCE / Outbox cancellation applies before claim. Remote undo is a non-goal.

RECEIPTS, NOT OMNISCIENCE

The agent gets a narrow command surface and correlated evidence. Receipts record operational fact, not semantic correctness.

SOURCE / src/agent-cli.ts / src/runtime/Runtime.ts / test/parity-process.test.ts

THE AGENT SURFACE — RUNNABLE

```
airlock-agent doctor
airlock-agent actions
airlock-agent schema process.run

airlock-agent run workflow.air --compact
airlock-agent runs --limit 10
airlock-agent run-receipt --plan-id pln_...
```

ABRIDGED EXECUTED OUTPUT — PROOF EXCERPT

```
{
  "schemaVersion": "airlock/program-run/v1",
  "profile": "compatibility",
  "workspace": "/tmp/airlock-proof/process/workspace",
  "result": {
    "state": "succeeded",
    "result": {
      "selected": "2:needle second\n",
      "transformed": "2:NEEDLE SECOND\n",
      "literal": "literal; touch should-not-exist",
      "states": [
        "succeeded",
        "succeeded",
        "succeeded"
      ]
    },
    "counts": {
      "plans": 3,
      "actions": 3,
      "artifacts": 6
    }
  }
}
```

PROJECTION BOUNDARY

This is a selected field projection from captured compact CLI output, not a standalone Schema-valid document. Program success still requires inspecting nested process state and outcome.

[OK] SUCCEEDED

All requested current actions completed and returned evidence.

[X] FAILED

Typed failure with completed records retained.

[~] PARTIAL

Earlier actions remain complete; no implied rollback.

REPLAY RETURNS RECEIPTS

Running the same run id again returns the recorded receipt for each call that already ran, instead of making it again. A file changed since stops the run.

EVIDENCE BOUNDARY

Receipts show admitted, attempted, observed, staged, or changed work. They do not prove that opaque code fulfilled intent.

EVIDENCE / A durable receipt for every possible crash point remains an acceptance gate.

WHAT THE EVIDENCE EARNS

Airlock already covers meaningful agent scripting shapes. Strong shell-replacement confidence still depends on representative workload and adversarial evidence.

SOURCE / README.md / docs/acceptance.md / docs/evidence/parity-50.md

REPRODUCIBLE REPOSITORY GATES

EVIDENCE COMMANDS

```
1 bun run verify
2 bun scripts/prove-vouch.ts
3 bun scripts/prove-vouch-operations.ts
4 bun scripts/prove-parity-50.ts
```

VITEST FILES	VITEST TESTS	PARITY PROGRAMS	REPEAT EXECUTIONS
52	255	11	50
+ 1 skipped	+ 16 skipped	checked-in current set	10 cases x 5

WHAT WORKS COMPREHENSIVELY ENOUGH TO STUDY

Structured file work, captured processes, artifact pipelines, bounded control, native edits, archives, local Git, builds, recoverable recursive removal, and staged HTTP intent.

WHAT REMAINS BEFORE A STRONG CLAIM

Representative model-generated tasks, direct shell A/B evidence, wider crash/overlap injection, hostile closure tests, labels, endpoint brokerage, real remote Vouch, signing, and release provenance.

> A STRONG FOUNDATION IS VISIBLE. THE RELEASE CLAIM REMAINS A GATE, NOT A SLOGAN.



READ NEXT

DESIGN.md / docs/macos-v1.md / docs/security-model.md
docs/acceptance.md / examples/parity / examples/vouch